Лекція 4
РЕГУЛЯРНІ ВИРАЗИ

Синтаксис, r-рядки, модуль r

Групування

Ми вже бачили, що у класі re.Match існує метод group(), який дозволяє отримати підрядок, який відповідає певному шаблону. Досі ми викликали цей метод без аргументів, проте логічно припустити із назви методу, що він насправді дозволяє працювати з групами рядків, що відповідають шаблону.

Для утворення групування слід взяти у круглі дужки () ту частину шаблону, з якої потрібно утворити групу.

Наприклад, щоб створити шаблон дати у форматі dd/mm/YYYY з виділенням підгруп для днів, місяці та років, можна скористатись таким регулярним виразом:

(\d{2})\/(\d{2})\/(\d{4})

Групування

Круглі дужки також використовуються для звичайного об’єднання частин шаблону при визначенні варіантів.

У випадку, якщо потрібно об’єднати декілька частин шаблону без явного виділення підгруп, потрібно після дужки (, що відкриває послідовність, вказати знак питання та двокрапку:

(?: шаблон)

Групування

Як ми вже згадували, для того, щоб повернути виділену підгрупу можна скористатись методом group(), вказуючи в якості аргументу порядковий номер шуканої підгрупи у шаблоні (нумерація підгруп починається з 1).

Наприклад, для шаблону дати **date=(\d{2})\/(\d{2})\/(\d{4})** команда date.group(1) поверне день, а date.group(2) – місяць і date.group(3) – рік.

У випадку, якщо потрібно отримати значення всіх шаблонів одночасно, можна скористатись методом groups(), який повертає кортеж з усіма підгрупами знайденими за шаблоном.

Групування

На основі задачі про навчальний заклад, що була розглянута на минулій лекції. Розглянемо такий приклад.

Декілька навчальних закладів у текстових файлах збирають інформацію про своїх студентів.

У файлах містяться дані про: ПІП студента, номер студентського квитка, дату народження, телефон.

Побудувати словник, ключами якого є номери студентських квитків, а значеннями – відповідна інформація про студентів.

Перевірити правильність інформації.

Відомо, що кожен заклад записує дату народження у різному форматі. Можливі очікувані формати даних у файлах:

01.01.2005

01/01/2005
2005-01-01

Розв’язок.

Зміна рядків за допомогою функцій та методів класу re.Pattern

Можливості модуля re не обмежуються лише простим знаходженням шаблонів у тексті. За допомогою його функцій та методів можна також здійснювати зміни у рядках згідно з шаблоном регулярного виразу.

Функція (метод)	Опис дії
re.sub(pattern, repl, string, count=0, flags =0) АБО **rgx.sub(repl, string, count=0)**	Змінює у рядку **string** усі рядки, що відповідають шаблону, на **repl**. Параметр **count** відповідає за кількість замін, якщо він не 0. аргумент **repl** може бути рядком, або функцією, що має один аргумент з класу re.Match і повертає змінений підрядок.
re.split(pattern, string, maxsplit=0, flags =0) АБО **rgx.split(string, maxsplit=0)**	Дозволяє розділити рядок **string** на підрядки, використовуючи у якості рядків-роздільників підрядки, що відповідають шаблону. У результаті повертає список підрядків, що утворились після поділу. Параметр **maxsplit** (якщо не 0) вказує на максимальну кількість рядків, що може бути утворена у результаті поділу.

Зміна рядків за допомогою функцій та методів класу re.Pattern

Легко зауважити, що sub() та split() діють схожим чином, як і методи рядків replace() і split(), відповідно.

Але функції модуля re є потужнішими та гнучкішими за своїх відповідників для рядків.

Наприклад

re.split(r`aieouAIEOU’, line)

розіб’є рядок line по латинських голосних літерах.

Зміна рядків

Повернемось знову до задачі про навчальні заклади. Нехай тепер крім даних з уже відомих нам ЗВО ми отримали дані з нових зкладів, які ставляться до заповнення своєї бази студентів не дуже ретельно.

Дата може бути записана у форматах dd.mm.yyyy, dd/mm/yyyy, yyyy-mm-dd

Також день та/або місяць можуть складатись з однієї цифри замість двох.

Трапляються записи, де номер телефону вказаний без вказання знаку +, або зовсім без коду країни - +380.

Зміна рядків

На основі попередньої програми потрібно:

- Привести шаблони для перевірки дати до відповідності;
- Звести всі телефонні номери до єдиного формату +380 (00) 000 0000.
- Звести дати єдиного формату dd.mm.yyyy.
- З метою пришвидшення роботи програми не викликати помилку при знаходженні неправильного запису, а записувати такі дані у окремий лог-файл.
- Відфільтрований словник з правильними даними записати у файл.

РОЗВ’ЯЗОК.

ІМЕНОВАНІ ПІДГРУПИ

Для зручності, підгрупам у шаблонах можна задавати кастомні імена. Для цього потрібно у кругрлих дужках написати знак запитання, символ P та власне ім’я у кутових дужках:

(?P<name>)

Наприклад у шаблоні дати проіменуємо підгрупи дня, місяця та року:

(?P<day>\d{2})\/(?P<month>\d{2})\/(?P<year>\d{4})

Якщо шаблон містить іменовані підгрупи, то у методі group() об’єкту класу re.Match можна отримати підрядок, що відповідає підгрупі, звернувшись за ім’ям: mt.group(‘name’).

Що отримаємо у результаті виконання коду:

```
date_pat = r'(?P<day>\d{2}).(?P<month>\d{2}).(?P<year>\d{4})'
DATE = re.compile(date_pat)

text = '14.03.2025'

res = re.search(DATE, text)

print(res.group('day'))
```

Що отримаємо у результаті виконання коду:

```
date_pat = r'(?P<day>\d{2}).(?P<month>\d{2}).(?P<year>\d{4})'
DATE = re.compile(date_pat)

text = '14.03.2025'

res = re.search(DATE, text)

print(res.group('day’)) # 14
```

Приклад: Обробка конфігураційних файлів

Конфігураційний файл складається з рядків, що задають значення певних параметрів програм.

Кожен рядок файлу має вигляд **<ім’я>=<значення>**.

У більшості великих систем є конфігураційні файли, оскільки вони дозволяють вносити зміни до програми без її перепрограмування.

Зрозуміло, що програма повинна «передбачити», що її можна змінювати та аналізувати свої конфігураційні файли.

Необхідно написати модуль, що аналізує конфігураційний файл та повертає словник з параметрами, у якому імена є ключами, а значення – значеннями.

Обробка конфігураційних файлів

Для вирішення поставленої задачі скористаємось можливостями регулярних виразів.

Токенами (символами) називатимемо елементи конфігураційного файлу, а програму, яка їх виділяє, – токенізатором.

Для нашої задачі маємо такі токени:

- Ім’я (NAME)
- Знак дорівнює (EQ)
- Ціле число (NUM_INT)
- Дійсне число (NUM_FLOAT)
- Рядок (STRING)
- Пропуск (WS)
- Кінець рядка (EOL)
- Коментар (COMMENT)
- Інше (OTHER)

Обробка конфігураційних файлів

- Синтаксис конфігураційного файла є доволі простий, один рядок виглядає таким чином: **<ім’я>=<значення>**.
- Між ім’ям та значенням може стояти будь-яка скінченна кількість пропусків, а після значення – кінець рядка.
- Оскільки часто у Python-коді зустрічаються коментарі, то будемо передбачати їх наявність у кінці рядків конфігураційного файла (вважаємо, що коментарі починаються з символа #).
- Значення можуть бути рядками, цілими, або дійсними числами.
- Числа позначаються звичним чином, а рядки беруться у апострофи або подвійні лапки.
- Для перевірки синтаксичної правильності конфігураційного файлу нам достатньо проаналізувати два токени: поточний та наступний.
- Якщо перевірена пара токенів належить множині допустимих пар, то перевірку пройдено успішно. В іншому випадку – синтаксична помилка.

Обробка конфігураційних файлів

Задамо шаблони для токенів:

```
P_NAME = r'(?P<NAME>[A-Za-zA-Яа-яЇЄҐїєґі_]\w*)'  # шаблон для імені
P_EQ = r'(?P<EQ>=)'  # шаблон для знаку дорівнює
P_NUM_INT = r'(?P<NUM_INT>[\+-]?\d+)'  # шаблон для цілого числа
P_NUM_FLOAT = r'(?P<NUM_FLOAT>[\+-]?\d+\.\d*)'  # шаблон для дійсного числа
P_STRING = r'(?P<STRING>(["\'])(.*?)(["\']))'  # шаблон для рядка
P_WS = r'(?P<WS>\s*)'  # шаблон для пробілів
P_COMMENT = r'(?P<COMMENT>#.*)'  # шаблон для коментарів
P_EOL = r'(?P<EOL>[\n])'  # шаблон для кінця рядка \n
P_OTHER = r'(?P<OTHER>.+)'  # шаблон для інших можливих виразів
```

Обробка конфігураційних файлів

Об’єднаємо отримані шаблони у один шаблон для аналізу конфігураційних файлів:

```
P_CONFIG = '|'.join([P_NAME, P_EQ, P_NUM_FLOAT, P_NUM_INT, P_STRING, P_WS, P_COMMENT, P_EOL, P_OTHER])
```

Обробка конфігураційних файлів

Далі визначимо словник допустимих пар токенів та токени, які потрібно пропустити:

```
VALID_COUPLES = {'NAME': 'EQ',  # після імені може йти лише =
            'EQ': {'NUM_INT', 'NUM_FLOAT', 'STRING'},  # після = число або рядок
            'NUM_INT': {'EOL'},  # після числа лише \n
            'NUM_FLOAT': {'EOL'},  # після числа лише \n
            'STRING': {'EOL'}, # після рядка лише \n
            'EOL': {'NAME', 'EOL'},  # з нового рядка або ім'я, або \n
            'OTHER': set(),  # після інших символів - порожня множина
            }

IGNORE = {'WS', 'COMMENT'}  # множина токенів, яку треба пропустити
```

Обробка конфігураційних файлів

Наступним кроком вирішення даної задачі є створення генератора токенів – це генераторна функція, що за рядком, який складається із багатьох рядків файлу, виокремлює і повертає послідовність токенів.

Ця функція опирається на шаблон для виділення наступного токену, ігноруючи при цьому пропуски та коментарі.

Токени будемо представляти у вигляді іменованих кортежів (імпортуються з бібліотеки collections):

```
Token = namedtuple('Token', ['type', 'value’])
```

де type – це тип токену (‘NAME’, ‘EQ’ і т. п.), value – це підрядок, що є значенням токену.

ОБРОБКА КОНФІГУРАЦІЙНИХ ФАЙЛІВ

На відміну від попередніх розглядуваних нами задач, які були вирішені процедурною парадигмою, для задачі про обробку конфігураційних файлів доцільно використати комбіновану парадигму: процедурну + ООП.

Для цього створимо клас ConfigFileDict, який відповідатиме за зчитування вмісту конфігураційного файлу і за допомогою генератора токенів будуватиме словник, ключами якого є імена, а значеннями – значення, що містяться у самому файлі.

У конструкторі цього класу оголосимо словник (змінний об’єкт!) дефолтних значень параметрів. Він буде модифікуватися даними з конфігураційного файла.

Для того, щоб повернути словник реалізуємо метод – getconfig().

ОБРОБКА КОНФІГУРАЦІЙНИХ ФАЙЛІВ

Реалізуємо за допомогою інкапсуляції внутрішній метод _check_syntax, який перевірятиме синтаксичну правильність рядків та викликатиме SyntaxError у разі виникнення такого роду помилок у даних з конфігураційного файла.

<u>РОЗВ’ЯЗАННЯ.</u>

Зауваження та рекомендації

Не зважаючи на досить велику значимість регулярних виразів при обробці текстів, варто все таки тримати на контролі можливість їх непередбачуваної компіляції для різних можливих рядків.

Щоб зменшити ймовірність виникнення помилок та небажаних ситуацій, варто перевіряти код для всеможливих різних випадків, застосовувати виведення проміжних результатів (підрядків, що відповідають шаблону), аналізувати позиції виділених підрядків, коментувати складні шаблони.

Література для додаткового вивчення

1. Обіванцев О.В. Об’єктно-орієнтоване програмування. Курс на основі Python. Матеріали лекцій. – К., Основа, 2017
2. Felix Lopez, Victor Romero. Mastering Python Regular Expressions – 2014
3. Hellmann D. – The Python Standard Library by Example – 2011
4. http://www.python-course.eu

F & Q